Exactly-Once Delivery to Apache Iceberg

How OLake Go Guarantees Every Row Lands Once — With No External Coordinator

ABSTRACT

Change Data Capture (CDC) promises that every source change lands exactly once in the lakehouse. In practice, it does not: after a transient failure, a driver restarting from its last saved checkpoint re-reads an overlap window of already-processed changes, and without idempotent writes those rows land in Apache Iceberg a second time. Iceberg's append-only snapshot model does not auto-deduplicate.

This paper presents OLake Go's exactly-once protocol, which requires no external coordinator — no Kafka Connect, no Debezium, no Flink. Exactly-once is built into the write path through four mechanisms: (1) stable row identity (`_olake_id` , `_op_type` , `_olake_timestamp`); (2) upsert-mode equality and positional deletes committed with the data; (3) snapshot-to-log handoff, where the committed Iceberg `SnapshotId` is written into a Pebble stream index (`_olake_id` → `file:position`); and (4) crash recovery that restores the checkpoint, re-reads the overlap window, and collapses duplicates via delete + insert so only net-new rows are committed.

Verified against the OLake Go source code and published benchmarks: Postgres CDC at 55,555 RPS (2× Fivetran), 4.01B rows full-load in 1.91 hrs for <$6.

1. Introduction

CDC captures change events from a source database's WAL/binlog/oplog. After a transient failure, the driver resumes from its last saved checkpoint and re-reads the changes in the overlap window — including rows already written to Iceberg. Without idempotent writes, the same row lands twice. Iceberg tables are append-only by snapshot and do not auto-deduplicate; achieving exactly-once requires delete files (equality or positional) or MERGE INTO logic coordinated with the ingest stream.

Most managed ELT tools sidestep this by writing to a warehouse and relying on warehouse upsert, or by inserting an external coordinator (Kafka Connect + Debezium, or Flink) between source and sink. That coordinator becomes the source of truth for exactly-once — but it is another system to run, monitor, and pay for.

The core tension: exactly-once to Iceberg is either outsourced to an expensive external coordinator, or it is not guaranteed at all. OLake Go takes a third path: exactly-once inside the driver, no coordinator, no lock-in.

2. OLake Go Architecture

OLake Go is a single Go binary. It reads WAL/binlog/oplog natively per source (Postgres pgoutput, MySQL binlog, MongoDB oplog, MSSQL CDC), normalizes every record into `_olake_id` , `_op_type` , `_olake_timestamp` , and writes through an Arrow-based writer to an embedded Java Iceberg server over gRPC. Data lands as Iceberg files in object storage (S3, GCS, Azure, MinIO) under catalogs including AWS Glue, Hive HMS, JDBC, and REST (Nessie, Apache Polaris, Unity Catalog, Lakekeeper, AWS S3 Tables).

No Spark, no Flink, no Kafka, no Debezium. The exactly-once protocol runs inside the OLake Go process itself.

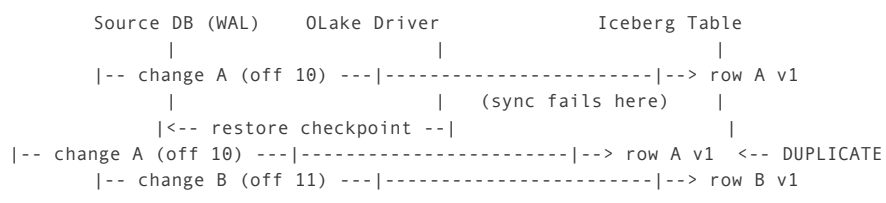


Figure 1 — The exactly-once gap. CDC failure → checkpoint restore → overlap re-read → duplicate rows in Iceberg (without idempotent writes).

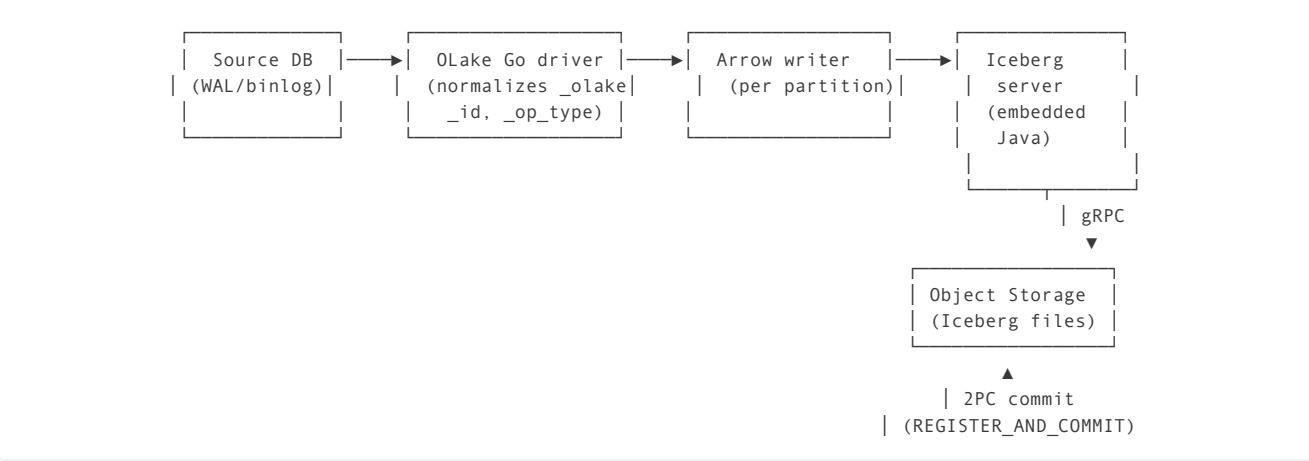


Figure 2 — OLake Go data-flow architecture. Source → driver → Arrow writer → Iceberg server → object storage. The 2PC commit is orchestrated by the driver; no external coordinator is in the path.

3. The 2PC Commit Mechanism

OLake Go uses a **two-phase commit** inside the writer:

Phase 1 — Stage. Data files, equality-delete files, and positional-delete files are staged to an `_olake_2pc/` prefix in object storage. A shared `finish.json` marks the commit point and is written idempotently, so a replay after a partial failure sees the same finish state and does not duplicate work.

Phase 2 — Commit. The driver sends a `REGISTER_AND_COMMIT` request to the Iceberg server. The server registers all file metadata and commits a single Iceberg snapshot atomically. Files are committed in a deterministic order: **equality deletes → data → positional deletes**, so readers always see a consistent state.

Upsert mode & deduplication. In upsert mode (non-append), every record carries `_olake_id`. The Arrow writer tracks row positions per partition:

- **First occurrence** of an `_olake_id` → an equality-delete entry is emitted, and the position is recorded.
- **Duplicate occurrence** → the previous position is emitted as a positional delete (n-1 logic); only the latest occurrence is kept.

The result: even if the same `_olake_id` arrives across multiple data files or retries, the committed Iceberg snapshot contains the row exactly once.

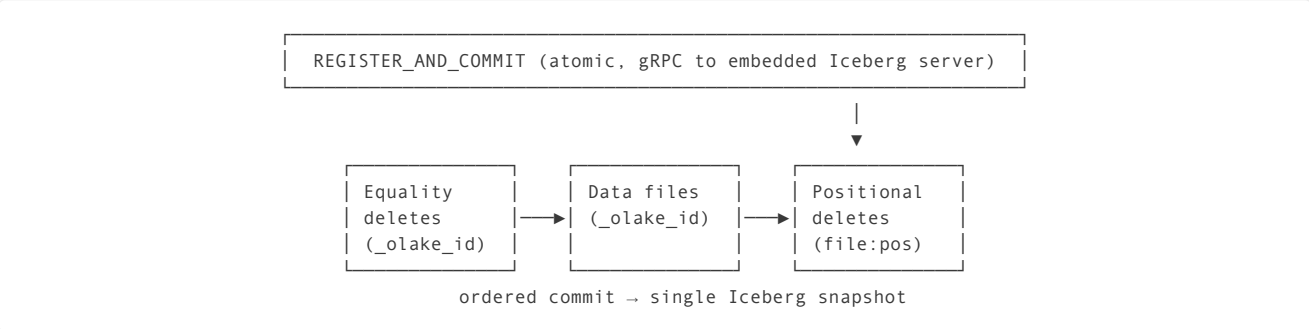


Figure 3 — 2PC commit workflow. Stage → equality/positional deletes → REGISTER_AND_COMMIT → Iceberg snapshot.

DELETE TYPE	PAYLOAD	PURPOSE
Equality delete	`_olake_id`	Marks all rows with this id for removal
Positional delete	`file:position`	Removes a specific occurrence of a duplicate id

Figure 4 — Equality vs positional delete. Equality deletes carry only `_olake_id`; positional deletes carry `file:position`. Duplicates collapse via n-1 logic: latest occurrence kept, previous position deleted.

Honest boundary. OLake's exactly-once is an **exactly-once visible result at the Iceberg table level** (idempotent writes plus dedup on recovery). It is not distributed-systems exactly-once messaging with an external transaction coordinator. We state this plainly.

4. Snapshot-to-Log Handoff & Recovery Index

The committed Iceberg `SnapshotId` is not discarded after commit. OLake writes it back into a **Pebble-based stream index** (`pkg/indexdb/pebble.go`) that maps `_olake_id` → `file:position`. The committed snapshot therefore becomes the durable recovery log: the index knows exactly which snapshot and file position holds each row.

The `_olake_2pc_state` table property carries committed chunk IDs / cursor positions and is refreshed on every retry, so a restarted writer always sees the up-to-date committed state rather than a stale one.

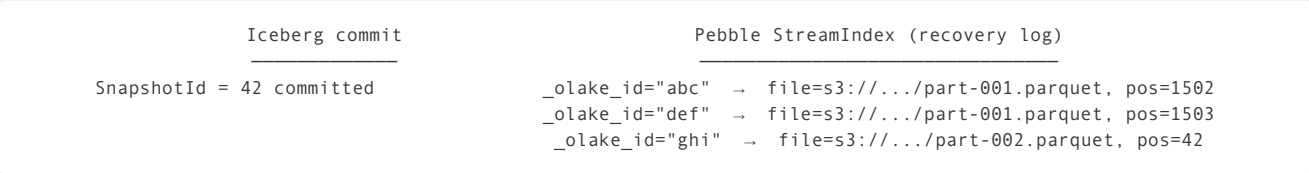


Figure 5 — Snapshot-to-log handoff. Committed Iceberg SnapshotId → Pebble StreamIndex entry `_olake_id` → `file:position`.

Every `REGISTER_AND_COMMIT` returns the `SnapshotId`, which `TableIndex.Commit(batch, committedSnapshotID)` applies to the index. On restart, the driver reads `_olake_2pc_state` from the table and resumes from the committed cursor — never reprocessing already-committed chunks.

5. Crash Recovery: Overlap-Window Dedup

When a sync fails mid-flight, OLake's recovery works as follows:

1. **Save checkpoint.** Before processing a batch, the driver persists state (a Pebble state file) marking the pre-insert cursor.
2. **Crash.** A failure occurs after some rows are written but before state is safely advanced.
3. **Restore.** On restart, state is restored to the last committed checkpoint.
4. **Re-read overlap.** The cursor re-reads the overlap window — including rows already committed to Iceberg.
5. **Dedup via delete + insert.** Duplicate `_olake_id`s arriving in the re-read are collapsed: the original row gets a delete file; only the net-new row is inserted.

This behavior is verified by OLake's 2PC integration tests (`tests/testutils/integration/2pc.go`). In the `insert_2pc` recovery scenario, after restoring state and re-syncing, only the `insert_2pc` row is visible (`expectedRowCountByOpType: 1`) — the original insert is deduplicated. After a subsequent normal sync, both the original and `insert_2pc` rows appear, each unique by `_olake_id` (`expectedRowCountByOpType: 2`).

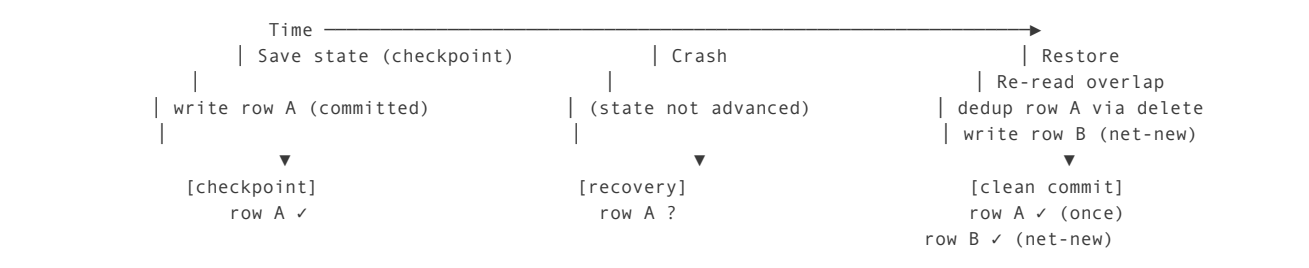


Figure 6 — Crash-recovery timeline. Checkpoint → crash → restore → re-read overlap → dedup → net-new commit.

BEFORE RECOVERY	AFTER RECOVERY (DEDUPED)

row A v1 (offset 10)	row A v1
row A v1 (offset 10) ← duplicate	(deleted via equality/positional delete)
row B v1 (offset 11)	row B v1

Figure 7 — Before/after dedup. Duplicate rows collapse to one visible row via delete + insert.

Visible result: exactly one row A, exactly one row B — exactly once.

6. Evidence: Benchmarks & Comparison

OLake Go's exactly-once claims are backed by published benchmarks (`olake.io/docs/benchmarks/ingestion/`).

TOOL	CDC THROUGHPUT	POSTGRES FULL-LOAD	COST (4.01B ROWS)	EXTERNAL CO-ORDINATOR	EXACTLY-ONCE TO ICEBERG
OLake Go	**55,555 RPS** (Postgres)	**580,113 RPS**	**<$6**	**None**	Built-in
Airbyte	~457 RPS	—	$5,560	Managed platform	Warehouse upsert
Fivetran	—	46,395 RPS	$5,560	Managed platform	Warehouse upsert
Debezium	—	—	Self-hosted	Kafka Connect required	Downstream logic
Estuary	—	—	$75	Flow collections	Downstream logic

> **KEY RESULT**
> 4.01B Postgres rows → Iceberg in 1.91 hrs for <$6 (vs Fivetran's $5,560, ~1,270× slower); Postgres CDC 55,555 RPS (2× Fivetran); MySQL CDC 59,951 RPS (1.52× Fivetran).

Benchmarks measure committed, deduplicated rows only; failed or retried attempts are rolled back so throughput reflects data that actually lands in Iceberg.

7. Operations & Deployment

OLake Go is self-hosted entirely within the customer's VPC or on-prem environment — critical for regulated industries that cannot compromise on data residency. Deploy via Docker Compose, standalone Docker, Helm/Kubernetes, or Airflow.

Catalogs: AWS Glue, Hive HMS, JDBC, REST (generic, Nessie, Apache Polaris, Unity Catalog, Lakekeeper, AWS S3 Tables).
Query engines: Athena, Trino, Spark, Snowflake, Databricks, ClickHouse, StarRocks, Dremio, DuckDB, and more — Iceberg tables are open and queryable by any engine.

8. Conclusion

OLake Go proves that exactly-once delivery to Apache Iceberg does not require an external coordinator. By combining stable row identity (`_olake_id`), upsert-mode equality and positional deletes, snapshot-to-log handoff into a Pebble recovery index, and overlap-window dedup on crash recovery, OLake Go guarantees every row lands once — using only its own process — at 55,555 RPS CDC with sub-dollar infrastructure cost for billion-row loads.

OLake publishes this and all technical content ungated, open source, and source-verifiable — a deliberate contrast to form-gated competitor white papers. Engineers are invited to review the implementation at `github.com/datazip-inc/olake`, run the benchmarks, and try the quickstart at olake.io.

References

1. OLake Go repository — github.com/datazip-inc/olake (Apache 2.0)
2. OLake Go product & benchmark docs — olake.io/olake-go/, olake.io/docs/benchmarks/ingestion/
3. Apache Iceberg Table Format Specification — iceberg.apache.org
4. OLake 2PC integration tests — olake/tests/testutils/integration/2pc.go
5. OLake Branding Guidelines — olake.io/branding/
6. OLake Fusion (table maintenance) — github.com/datazip-inc/olake-fusion